

AI APIs

The practical landscape of how you actually connect to models — OpenAI API, Anthropic API, Google Gemini API, OpenRouter, Groq, Together AI, and the Hugging Face Inference API.

DIFFICULTY LEVEL

Beginner-Intermediate

ESTIMATED PRACTICE TIME

80-100 minutes

ESTIMATED READING TIME

50-65 minutes

PREREQUISITES

Modules 01-11

LEARNING OUTCOMES

By the end of this module, you will be able to: connect to any major provider's API directly; understand what aggregator platforms like OpenRouter offer over direct provider APIs; choose an inference platform based on speed, cost, or open-model needs; and design your code so switching providers later doesn't require a rewrite.

Table of Contents

1.	Introduction	
2.	Before You Begin	
3.	Main Lesson	
3.1	OpenAI API	
3.2	Anthropic API	
3.3	Google Gemini API	
3.4	OpenRouter	
3.5	Groq	
3.6	Together AI	
3.7	Hugging Face Inference API	
4.	Visual Learning	
5.	Real World Applications	
6.	Practical Examples	
7.	Code Examples	
8.	Common Mistakes	
9.	Best Practices	
10.	Hands-on Activities	
11.	Mini Project	
12.	Review Questions	
13.	Cheat Sheet	
14.	Key Takeaways	
15.	Additional Resources	

1. Introduction

What is this topic?

Module 11 covered which model to use. This module covers how you actually reach it — the direct provider APIs, plus a growing layer of aggregator and specialized inference platforms that sit between you and the underlying models.

Why is it important?

The same model can often be accessed multiple ways — directly from its provider, through an aggregator, or via a specialized fast-inference platform — each with different tradeoffs in cost, speed, reliability, and flexibility.

Your Code



Direct API or Aggregator



Underlying Model

Why should AI Engineers learn it?

Knowing this landscape lets you make deliberate infrastructure decisions — when to go direct to a provider, when an aggregator saves engineering time, and when a specialized fast-inference platform is worth adopting for latency-critical products.

Where is it used in the real world?

Startups experimenting with multiple models often start on an aggregator like OpenRouter for flexibility, then move to direct provider APIs at scale for cost or reliability reasons; latency-critical products often adopt specialized fast-inference platforms like Groq.

Why is it relevant today?

The API and inference layer has grown into its own ecosystem alongside the models themselves — aggregators, fast-inference specialists, and open-model hosting platforms are now standard parts of the AI engineering toolkit, not niche options.



Warning

As with Module 11, specific pricing, rate limits, and supported models per platform change frequently. This module focuses on each platform's general role and tradeoffs — verify exact current details on official documentation before making production decisions.

2. Before You Begin

RECAP

Model comparison (Module 11) — this module assumes you've already thought about which model(s) you want; now it's about how you reach them.

RECAP

Cost/latency optimization (Module 09) — API/platform choice is another lever on both, alongside the model choice itself.



Tip

Whichever API or platform you start with, keep your model-calling code behind a thin wrapper function/interface — it makes switching or comparing providers later far less painful (Section 7 shows this pattern).

3. Main Lesson

Each API/platform follows: **Definition** → **Simple Explanation** → **Analogy** → **Strengths** → **Considerations** → **Best Use Cases** → **Common Mistakes** → **Summary**.

3.1 OpenAI API

DEFINITION

The OpenAI API is OpenAI's direct, official interface for accessing GPT models programmatically, including chat completions, tool use, and structured outputs.

SIMPLE EXPLANATION

As one of the earliest widely-adopted LLM APIs, it has an extensive ecosystem of SDKs, third-party libraries, and community examples built around it.

💡 *Like going straight to the manufacturer for a part — you get official support and the full current feature set, without any intermediary layer.*

STRENGTHS

Direct access to the latest GPT capabilities, mature SDKs across languages, large community/documentation base.

CONSIDERATIONS

Locks you into OpenAI specifically unless you build an abstraction layer (Section 7); rate limits and pricing tiers apply as with any direct provider API.

BEST USE CASES

Products built specifically around GPT's capabilities, or teams already invested in OpenAI's broader tooling ecosystem.

COMMON MISTAKES

Hardcoding OpenAI-specific request/response formats deep throughout an application, making a future provider switch costly.

SUMMARY

The OpenAI API is the direct, official route to GPT models — mature and well-documented, best paired with an abstraction layer if provider flexibility matters.

3.2 Anthropic API

DEFINITION

The Anthropic API is Anthropic's direct, official interface for accessing Claude models — the API used throughout this entire course's code examples.

SIMPLE EXPLANATION

It provides the Messages API (used in every code example so far), including tool use, structured outputs, streaming, and extended thinking / reasoning-mode access for supported models.

💡 *Like the official service center for a specific car brand — deepest, most current support specifically for Claude's full feature set, including capabilities not always mirrored elsewhere first.*

STRENGTHS

Direct access to the latest Claude capabilities as soon as they ship, official SDK support (Python, JavaScript/TypeScript), strong documentation.

CONSIDERATIONS

Provider-specific request/response format (though broadly similar in spirit to other chat-

completion APIs); as with any direct API, ties you to Anthropic unless abstracted.

BEST USE CASES

Any product built around Claude's specific strengths (Module 11, 3.2) — coding tools, long-document analysis, structured/XML-heavy prompting workflows.

COMMON MISTAKES

Not checking for newly released features (e.g., new tool-use capabilities) since direct APIs often get new capabilities before aggregators catch up.

SUMMARY

The Anthropic API is the direct, official route to Claude — the interface this entire course's code examples have been built on.

🌐3.3 Google Gemini API

DEFINITION

The Google Gemini API (via Google AI Studio or Vertex AI) is Google's direct interface for accessing Gemini models, including their native multimodal capabilities.

SIMPLE EXPLANATION

It offers two main access paths: Google AI Studio (simpler, more consumer/developer-friendly) and Vertex AI (enterprise-grade, deeper Google Cloud integration) — a choice worth understanding before starting.

💡 *Like choosing between a simple retail storefront and a full enterprise account with a supplier — both sell the same core product, but with very different levels of infrastructure integration and support.*

STRENGTHS

Direct access to Gemini's multimodal capabilities, tight integration with Google Cloud infrastructure via Vertex AI for enterprise needs.

CONSIDERATIONS

Two different access paths (AI Studio vs. Vertex AI) with different setup complexity — worth choosing deliberately based on your project's scale and existing infrastructure.

BEST USE CASES

Multimodal applications, and teams already using Google Cloud infrastructure who benefit from Vertex AI's deeper integration.

COMMON MISTAKES

Choosing Vertex AI's enterprise complexity for a small project that would be better served by the simpler Google AI Studio path, or vice versa.

SUMMARY

The Gemini API offers two access paths suited to different project scales — direct route to Gemini's multimodal strengths (Module 11, 3.3).

🔗3.4 OpenRouter

DEFINITION

OpenRouter is an aggregator platform that provides a single, unified API to access many different models from many different providers, without needing separate accounts/SDKs for each.

SIMPLE EXPLANATION

Instead of integrating OpenAI's SDK, Anthropic's SDK, and Google's SDK separately, you integrate OpenRouter once and can switch between (or compare) models from many

providers using one consistent interface.

💡 *Like a universal remote control that works with every brand of TV — one interface, many underlying devices, instead of needing a separate remote for each one.*

STRENGTHS

Single integration point for many models/providers, easy to A/B test or switch models, often exposes newer or niche open-weight models quickly.

CONSIDERATIONS

Adds a layer between you and the provider (potential latency/reliability implications); may lag slightly behind direct APIs for brand-new provider-specific features.

BEST USE CASES

Early-stage products comparing multiple models before committing, or products wanting easy flexibility to switch models without major code changes.

COMMON MISTAKES

Assuming an aggregator always has 100% feature parity with a model's direct API — provider-specific advanced features sometimes arrive on the aggregator later.

SUMMARY

OpenRouter provides one unified interface across many providers — valuable for flexibility and comparison, at the cost of a thin intermediary layer.

⚡3.5 Groq

DEFINITION

Groq is a specialized inference platform built around custom hardware (LPUs — Language Processing Units) designed specifically for extremely fast LLM inference speed.

SIMPLE EXPLANATION

Groq doesn't train its own models — it hosts existing open-weight models (like Llama) on purpose-built hardware optimized for very low-latency, high-throughput generation, often dramatically faster token generation than typical GPU-based inference.

💡 *Like a specialty print shop with equipment built specifically for extremely fast turnaround — they're not the publisher, but they can produce the same content far faster than general-purpose printing equipment.*

STRENGTHS

Very low latency / high token-generation speed, competitive pricing for the speed offered, straightforward API for supported open-weight models.

CONSIDERATIONS

Limited to the specific set of (mostly open-weight) models Groq hosts — not a source for accessing closed models like GPT or Claude directly.

BEST USE CASES

Latency-critical, interactive applications (real-time voice assistants, live coding tools) where generation speed directly impacts user experience.

COMMON MISTAKES

Expecting Groq to host closed-source models (GPT, Claude, Gemini) — it's specifically for the open-weight models it supports.

SUMMARY

Groq trades model selection breadth for exceptional inference speed on the open-weight models it supports — a strong fit for latency-critical use cases.

3.6 Together AI

DEFINITION

Together AI is a platform providing managed inference and fine-tuning infrastructure for open-weight models, letting teams use models like Llama, Mistral, or Qwen without managing their own GPU infrastructure.

SIMPLE EXPLANATION

It sits between "fully self-hosted open-weight model" and "fully closed API" — you get open-weight model flexibility (choice of model, potential fine-tuning) with managed infrastructure convenience (no need to run your own servers).

 *Like leasing a car instead of buying one outright or renting hourly — you get more control and customization than a pure rental, without the full responsibility of ownership and maintenance.*

STRENGTHS

Access to a wide range of open-weight models without self-hosting, fine-tuning infrastructure included, generally competitive pricing for open-model access.

CONSIDERATIONS

Limited to open-weight models (not closed models like GPT/Claude/Gemini); still a third-party dependency, even if lighter-weight than full self-hosting.

BEST USE CASES

Teams wanting open-weight model flexibility and fine-tuning capability without taking on full infrastructure/ML ops responsibility themselves.

COMMON MISTAKES

Choosing full self-hosting when a managed platform like Together AI would meet the same open-weight needs with far less operational overhead.

SUMMARY

Together AI offers managed access to open-weight models — a middle ground between full self-hosting and closed-API-only options.

3.7 Hugging Face Inference API

DEFINITION

The Hugging Face Inference API provides hosted access to a vast range of models (LLMs and beyond — embeddings, classification, image models) from the Hugging Face Hub, the largest open-source model repository.

SIMPLE EXPLANATION

Beyond just LLMs, Hugging Face's ecosystem spans nearly every kind of ML model — making its Inference API useful whenever a project needs more than just chat-style LLM calls (e.g., embeddings for RAG, Module 07, or classification models).

 *Like an enormous public library with an on-demand printing service — an immense catalog of available "books" (models) of every kind, with a service to access them without hosting your own copy.*

STRENGTHS

Enormous model catalog spanning far beyond LLMs (embeddings, classification, vision), strong open-source community and documentation, useful single access point for varied ML needs.

CONSIDERATIONS

Performance/latency for hosted inference can vary by specific model and hosting tier; production-scale needs may require dedicated hosting options rather than shared inference endpoints.

BEST USE CASES

Projects needing non-LLM models (embeddings for RAG, classifiers) alongside LLM calls, or experimentation across a very wide range of open-source models.

COMMON MISTAKES

Overlooking Hugging Face for non-chat needs (like embeddings, Module 07, 3.1) when it's often one of the most convenient sources for exactly that.

SUMMARY

The Hugging Face Inference API is the broadest-catalog option, valuable especially beyond pure chat LLM use cases, like embeddings and classification.

4. Visual Learning

📌 The API Landscape

Ways to Access Models

- **Direct Provider APIs** (*OpenAI, Anthropic, Gemini*)
- **Aggregators** (*OpenRouter — one API, many providers*)
- **Specialized / Open-Model Hosting** (*Groq — speed; Together AI — managed open models; Hugging Face — broad catalog*)

📊 Platform Comparison

Platform	Best Known For	Model Access
OpenAI API	Official GPT access, mature ecosystem	GPT family only
Anthropic API	Official Claude access, newest features first	Claude family only
Gemini API	Multimodal, Google Cloud integration	Gemini family only
OpenRouter	One API, many providers	Broad, multi-provider
Groq	Extremely fast inference	Select open-weight models
Together AI	Managed open-model hosting + fine-tuning	Open-weight models
Hugging Face	Broadest model catalog (LLM + beyond)	Very broad, open-source focused

🔄 Abstraction Layer Pattern



5. Real World Applications

Scenario	Common Platform Fit
Real-time voice assistant needing minimal latency	Groq (specialized fast inference)
Early-stage startup comparing several models	OpenRouter (single integration, easy switching)
RAG system needing embeddings + an LLM	Hugging Face (embeddings) + any LLM provider
Enterprise product deeply integrated with Google Cloud	Gemini API via Vertex AI
Team wanting open-model fine-tuning without owning GPUs	Together AI

6. Practical Examples

Beginner: Matching platform to need

For each: (1) a latency-critical live transcription assistant, (2) a small team wanting to compare 3 models quickly, (3) a project needing sentence embeddings for RAG — name the platform from Section 3 you'd consider first.

Beginner: Direct API vs. aggregator tradeoffs

List 2 advantages and 2 disadvantages of using OpenRouter instead of Anthropic's direct API for a Claude-based project.

Intermediate: Building a wrapper function

Write a simple wrapper function (in your language of choice) that takes a prompt and a "provider" argument, and routes the call to either OpenAI or Anthropic's SDK accordingly.

Advanced: Multi-platform architecture

Design a system that uses Groq for a latency-critical sub-task and a direct provider API for a complex reasoning sub-task, justifying the split.

7. Code Examples

🐍Python — Provider-agnostic wrapper (abstraction layer pattern)

```
# pip install anthropic openai
import anthropic
import openai

anthropic_client = anthropic.Anthropic(api_key="ANTHROPIC_KEY")
openai_client = openai.OpenAI(api_key="OPENAI_KEY")

def generate(prompt, provider="anthropic", model=None, max_tokens=300):
    """One function, swappable provider — your app code never changes."""
    if provider == "anthropic":
        r = anthropic_client.messages.create(
            model=model or "claude-sonnet-4-6", max_tokens=max_tokens,
            messages=[{"role": "user", "content": prompt}]
        )
        return r.content[0].text
    elif provider == "openai":
        r = openai_client.chat.completions.create(
            model=model or "gpt-4o", max_tokens=max_tokens,
            messages=[{"role": "user", "content": prompt}]
        )
        return r.choices[0].message.content
    else:
        raise ValueError(f"Unknown provider: {provider}")

# Your application code stays identical regardless of provider:
print(generate("Explain RAG in one sentence.", provider="anthropic"))
print(generate("Explain RAG in one sentence.", provider="openai"))
```

🌐JavaScript — Using OpenRouter's unified API

```
// OpenRouter uses an OpenAI-compatible request format for many models
async function callOpenRouter(model, prompt) {
    const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
        method: "POST",
        headers: {
            "Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
            "Content-Type": "application/json",
        },
        body: JSON.stringify({
            model, // e.g. "anthropic/claude-sonnet-4.6" or "openai/gpt-4o"
            messages: [{ role: "user", content: prompt }],
        }),
    });
    const data = await response.json();
    return data.choices[0].message.content;
}

// Same function, different models, one integration
console.log(await callOpenRouter("anthropic/claude-sonnet-4.6", "Explain RAG in one sentence.));
console.log(await callOpenRouter("meta-llama/llama-3-70b", "Explain RAG in one sentence.));
```



Tip

The wrapper-function pattern in the Python example is worth adopting even if you only use one provider today — it's cheap insurance against a costly rewrite if you ever need to switch or add a second provider.

8. Common Mistakes

- **Hardcoding one provider's request/response format** throughout an application, making future switches costly.
- **Assuming aggregators always have full feature parity** with direct provider APIs — new features sometimes arrive on direct APIs first.
- **Expecting specialized platforms like Groq** to host closed-source models they don't support.
- **Choosing full self-hosting** when a managed platform like Together AI would meet the same needs with less overhead.
- **Overlooking Hugging Face for non-chat needs** like embeddings when it's often a convenient source.
- **Not verifying current pricing/rate limits** before committing to a platform for a production workload.

9. Best Practices

- **Build a thin abstraction layer** around model calls from day one, regardless of which provider you start with.
- **Choose direct APIs** when you need the newest features fastest or are committed to one provider long-term.
- **Choose aggregators (OpenRouter)** when comparing models or wanting easy flexibility.
- **Choose specialized platforms (Groq)** specifically when latency is a critical product requirement.
- **Choose managed open-model platforms (Together AI)** over full self-hosting unless you have genuine infrastructure needs.
- **Check current documentation** for pricing, rate limits, and supported models before committing at scale.

10. Hands-on Activities

Easy 5 exercises

1. List all 7 platforms from Section 3 and one distinguishing feature for each, from memory.
2. Explain the difference between a direct provider API and an aggregator in your own words.
3. For a latency-critical use case, name the platform you'd evaluate first and why.
4. Visit one platform's official docs and note today's supported models list.
5. List 2 reasons a team might choose Together AI over fully self-hosting an open-weight model.

Intermediate 3 exercises

1. Build the Python provider-agnostic wrapper from Section 7 and test it with 2 real providers.
2. Sign up for OpenRouter (or review its docs) and identify 3 models from different providers accessible through one API key.
3. Compare response latency for the same prompt on a direct provider API vs. Groq (if accessible), and note the difference.

Challenge 2 exercises

1. Design a full abstraction layer (interface + multiple implementations) supporting at least 3 of the platforms from this module, with a consistent function signature across all of them.
2. Research current pricing for the same underlying model (e.g., a Llama variant) across OpenRouter, Together AI, and Groq, and compare the cost/speed tradeoffs.

11. Mini Project

Goal

Build a small "provider router" tool that sends a prompt to the best-fit platform based on a simple priority flag (speed vs. quality vs. cost).

Requirements

- API access to at least 2 different platforms from this module (e.g., a direct provider API and OpenRouter or Groq)
- Python or JavaScript

Step-by-Step Instructions

1. Build wrapper functions for each platform, following the Section 7 pattern.
2. Write a router function that takes a prompt and a priority ("speed", "quality", or "cost") and picks the matching platform/model.
3. Test the same prompt through all 3 priority modes and compare the actual results, latency, and (if available) cost.
4. Print a summary table of which platform was used for each mode and its measured performance.

Expected Output

```
Priority: speed    → Routed to: Groq          → Latency: 0.3s
Priority: quality  → Routed to: Claude API    → Latency: 1.4s
Priority: cost     → Routed to: OpenRouter (cheap model) → Latency: 0.9s
```

Possible Improvements

- Add automatic fallback: if the primary platform fails or times out, retry via a secondary platform.
- Log actual measured cost per call to refine the "cost" routing decision over time.
- Extend the router to consider task type (e.g., always route embeddings calls to Hugging Face).

12. Review Questions

Multiple Choice (10)

1. What does OpenRouter primarily provide?

- A) Its own trained model B) A single unified API across many providers C) Only image generation D) Fine-tuning only

2. Groq is best known for:

- A) Training new foundation models B) Extremely fast inference via specialized hardware C) Being the cheapest option always D) Hosting closed-source models

3. Together AI primarily offers:

- A) Closed-model-only access B) Managed hosting and fine-tuning for open-weight models C) No API access D) Only embeddings

4. The Hugging Face Inference API is notable for:

- A) Only supporting chat LLMs B) An enormous catalog spanning LLMs, embeddings, and more C) Being closed-source only D) Requiring no account

5. Vertex AI is associated with which provider's Gemini access path?

- A) OpenAI B) Google, for enterprise-grade Gemini access C) Anthropic D) Meta

6. A key advantage of a direct provider API over an aggregator is:

- A) Always cheaper B) Access to the newest provider-specific features fastest C) No SDK needed D) Always faster

7. Why build a thin abstraction layer around model calls?

- A) It's required by every provider B) It makes switching or comparing providers later much easier C) It reduces token count D) It increases temperature

8. Groq would NOT be a good fit for:

- A) A latency-critical voice assistant B) Accessing a closed-source model like GPT directly C) Fast open-weight model inference D) Real-time applications

9. Which platform is most associated with embeddings and non-LLM model access?

- A) Groq B) Hugging Face C) OpenRouter D) Vertex AI only

10. What's a common mistake when using an aggregator platform?

- A) Assuming full feature parity with direct APIs at all times B) Using it to compare models C) Using one API key for multiple providers D) Testing multiple models easily

Identification (5)

1. _____ is an aggregator providing one unified API across many model providers.

2. _____ is a specialized inference platform known for extremely fast generation speed via custom hardware.

3. _____ offers managed hosting and fine-tuning infrastructure for open-weight models.

4. _____ provides access to an enormous catalog of models, including embeddings and classification, beyond just chat LLMs.

5. _____ is Google's enterprise-grade access path for Gemini, integrated with Google Cloud.

True or False (5)

1. OpenRouter trains its own foundation models.

2. Groq is designed specifically for very low-latency, high-throughput inference.

3. Direct provider APIs sometimes receive new features before aggregators do.

4. A thin abstraction layer around model calls makes switching providers easier later.

5. Hugging Face's Inference API is limited strictly to chat-style LLMs.

Situational (5)

- 1. You're building a real-time voice assistant where every millisecond of latency matters. Which platform would you evaluate first?**
- 2. Your small team wants to quickly compare 4 different models without integrating 4 separate SDKs. What platform fits this need?**
- 3. You need sentence embeddings for a RAG pipeline (Module 07). Which platform is a natural fit?**
- 4. Your enterprise is already heavily invested in Google Cloud infrastructure. Which Gemini access path would you consider?**
- 5. You want open-weight model flexibility and fine-tuning without managing your own GPU servers. What platform fits this need?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-A

Identification: 1-OpenRouter, 2-Groq, 3-Together AI, 4-Hugging Face Inference API, 5-Vertex AI

True or False: 1-False, 2-True, 3-True, 4-True, 5-False

Situational (sample reasoning):

1. Groq — specialized for extremely fast inference.
2. OpenRouter — one unified API across many providers.
3. Hugging Face Inference API — broad catalog including embeddings.
4. Vertex AI (Gemini's enterprise-grade Google Cloud path).
5. Together AI — managed open-model hosting and fine-tuning.

13. Cheat Sheet

■ Direct APIs

OpenAI, Anthropic, Gemini — newest features first, single-provider only.

✂ Aggregator

OpenRouter — one API, many providers, easy comparison/switching.

⚡ Specialized/Open

Groq — speed.

Together AI — managed open-model hosting.

Hugging Face — broadest catalog.

🔄 Golden Rule

Build a thin abstraction layer — cheap insurance against a costly future rewrite.

14. Key Takeaways

- Direct provider APIs (OpenAI, Anthropic, Gemini) offer the newest features fastest, but lock you into one provider unless abstracted.
- OpenRouter aggregates many providers behind one API — valuable for flexibility and comparison.
- Groq specializes in extremely fast inference for supported open-weight models — ideal for latency-critical products.
- Together AI offers managed open-weight model hosting and fine-tuning without full self-hosting responsibility.
- Hugging Face's Inference API spans the broadest model catalog, valuable especially beyond pure chat use cases like embeddings.
- A thin abstraction layer around model calls is cheap insurance against a costly rewrite if your platform needs change later.

15. Additional Resources

Official Documentation

- OpenAI — platform.openai.com/docs
- Anthropic — docs.claude.com
- Google — ai.google.dev (Gemini) and Vertex AI documentation
- OpenRouter — openrouter.ai/docs
- Groq — groq.com/docs (GroqCloud)
- Together AI — docs.together.ai
- Hugging Face — huggingface.co/docs (Inference API / Inference Endpoints)

Communities & Further Learning

- Each platform's changelog/release notes for staying current on new models and features as this landscape evolves quickly.